the loop is unrolled and then choose the appropriate vectorization pass. Alternatively, compilers could apply an unrolling or a re-rolling pass before the basic block based or strip-mining based vectorization pass respectively, and if the vectorization pass does not succeed, discard the re-rolled/unrolled results (if they are not profitable by themselves).

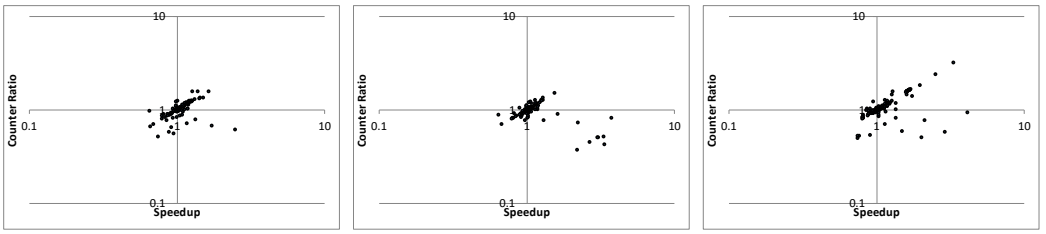
5.4 Unroll-and-Jam

Unroll-and-jam is primarily employed to facilitate data reuse by improving register usage, which decreases the number of memory accesses. In addition, it may enable vectorization on the outer loop without performing interchange [Nuzman and Zaks 2008]. Finally, while not as important, unroll-and-jam may reduce control overhead and/or increase ILP similarly to unrolling. Compilers may reverse a source level unroll-and-jam by applying an interchange → re-rolling → interchange sequence accordingly.

The correlation results for unroll-and-jam are rather interesting. ICC has a high negative correlation (-0.9) with `l1d.replacement` and has a moderate positive correlation (0.4) with `%l1_hit`, indicating that an improvement in L1 hit rate is a major factor for the performance gain. GCC and Clang, on the other hand, have lower negative correlations (-0.6∼-0.5) with `l1d.replacement`; however, they also have negative correlations (-0.6∼-0.5) with `inst_retired.any`. The main difference between ICC and GCC/Clang is the correlations with `l2_rw_rate`. For this metric, ICC exhibits a high negative correlation (-0.7) while GCC and Clang both have moderate positive correlation (0.3). These values imply that unroll-and-jam has a different effect on ICC compared to GCC/Clang.

Figure 6 contains plots of speedup (x-axis) vs. change in `l2_rw_rate` (y-axis) for all three compilers under the influence of unroll-and-jam. From the figure, we see that at low speedup/slowdown, all three compilers show positive correlations with the metric. In fact, 75% of ICC's data points land in quadrant 1 and 3, suggesting a positive correlation. However, for high speedup cases, ICC shows decrease in `l2_rw_rate`, represented by the points in quadrant 4. Since the Pearson correlation biases towards data points with higher values, the correlation coefficient becomes negative. In order to understand ICC's opposite correlation with `l2_rw_rate` at different speedup ranges, we inspected other metrics and found that:

(1) At high speedup range, `l1d.replacement` is significantly reduced, which means that the speedup is achieved from better data reuse and thus fewer L1 eviction.
(2) At low speedup/slowdown range, the performance change is due to other factors such as lower control overhead and/or higher ILP.



(a) GCC (b) ICC (c) Clang

Fig. 6. Speedup vs. change in `l2_rw_rate` due to unroll-and-jam

On the contrary, GCC and Clang have fewer points in quadrant 4 and more points showing positive correlation. More notably, Clang is able to obtain high speedup with a positive correlation with `l2_rw_rate`. By digging into other metrics, we learned that a majority of the speedup